Chapter 17. Designing for Resilience

By Vitaliy Shipitsyn
with Christof Leng, Garrett Holthaus, and Kieran Broadfoot

Resilience enhances reliability. *Reliability* is a system's ability to consistently operate within designed norms, meeting both functional and nonfunctional requirements. *Resilience*, on the other hand, is a system's ability to detect and diagnose active failure patterns automatically or with the help of trained people or AI agents. A resilient system uses predefined response plans to reduce the impact of failures, even for novel failures that were not anticipated during system design.

Resilience provides business continuity for the users who rely on your systems. A reliable system may consistently meet its SLO targets under normal conditions, but a resilient system maintains high levels of reliability in more adverse conditions. When a predefined response isn't prepared, that alone might burn through your system's error budgets. A lasting outage without a rebounding clean recovery can have an existential impact on business survivability. If you lose all of your users' data and cannot restore it, this may put your entire business at risk.

Your challenge is to anticipate unusual circumstances that stress systems beyond their original design expectations, testing their robustness. A system's availability, security, integrity, or performance characteristics might fail—independently or simultaneously, partially or fully, for benign or malicious reasons. The open-ended nature of this challenge highlights that resilience is also about engendering flexibility in people, processes, and technologies that support your systems.

In this chapter, we discuss a principled approach to understanding the failure modes that threaten the reliability of systems and designing resilience capabilities that support fulfilling user expectations. When you have a clear understanding of goals and risks, building mechanisms to control these risks becomes a structured engineering practice that adds durable resilience capabilities to complement, or gracefully extend, the reliability of your systems and teams.

Resilience can be built into:

- The *system itself*, so that it can automatically recover not only from unexpected failures but also from well-understood scenarios such as overload, failing dependencies, bad data, or parts of the system becoming unavailable
- The *operational workflows* supporting the system, so that on-callers are prepared for incident management, lending or borrowing expertise, and handling scenarios *not* fully covered by automated responses, such as rollbacks, drains, or throttling
- The *business continuity* of your organization's mission, through defense-in-depth mechanisms

Of course, no one can anticipate and prepare for handling all failures. As you read, remember that every design choice you make is ultimately a cost-benefit trade-off—part of sustainable adaptability to changing conditions after considering the threat landscape for your systems, teams, and organizations.

The Importance of Resilience

Resilience is the ability to handle severe failures that warrant some form of response, whether technical or human-led, as conditions worsen.

In practical terms, resilience means that failures are handled well enough that users do not experience a perceptible degradation of functionality, or, if degradation is unavoidable, the reduced functionality is clearly and unambiguously explained to the users.

In technical terms, resilience means maintaining adequate service quality while the application or its underlying infrastructure is experiencing some type of failure, even if that failure was not anticipated during system design.

Emerging Reliability

Let's consider how resilience enhances reliability over system lifetime.

Reliable systems deliver the promised reliability as long as the operating conditions are within designed or measured limits, but they could break outside those ranges (e.g., excessive traffic, invalid input, insufficient resources). You might discover or test some of the limits, for example with load testing or chaos engineering. *Resilient* systems, however, are prepared for conditions beyond the expected operating limits and tested to respond in a manner you define as appropriate.

Designing for resilience is the practice of prioritizing failures, extrapolating to novel conditions or combinations of failures, reducing their impact, and demonstrating mitigations

throughout the lifecycle of the system. When your findings and mitigations fuse into system designs, the system's reliability grows, and you can push the resilience frontier further.

Higher and Lower Probability

Resilience is actionable in three distinct ways: correctly and cost-effectively handling all higher-probability failures, prioritizing higher-impact (severity) failures first, and adequately handling the most impactful lower-probability failures. It can be tempting to focus on higher-probability failures and defer the lower-probability cases, but both require deliberate attention and commensurate investments.

Every resilience improvement has a cost and a point at which its benefits are realized. If those benefits are highly unlikely to ever be realized, it may be difficult to justify investing heavily in the improvement. Let's summarize how engineering investment often works in practice when what you want is less obvious than for higher-probability failures:

Lower-probability failures

Typically, reliability work in complex systems focuses on maximizing service availability at a high level of quality for an acceptable cost. For many organizations, a five-nines (99.999%) SLO is a practical upper limit (especially if users access the services through a lower-SLO system, such as the internet). Investing to close the remaining failures (~0.001%), especially when addressing them has a disproportionately high cost. Unfortunately, some of these failures could burn decades of error budget. If you don't consider them at all, you may overlook even cheap solutions. You don't always need the kind of high-quality solutions you design for higher-probability failures, but you should at least consider whether a low-cost mitigation—even if it only partially reduces the impact—is feasible.

Not worth the effort

After assessing low-probability and high-impact risks, you might conclude that any form of investment in them makes little sense because the available mitigation options don't sufficiently reduce the risk. That may be the right conclusion. However, consider periodically revisiting these deferred risks to reassess new options in the ever-changing technical landscape.

Importantly, investment in resilience may not always result in a technical implementation; instead, it might involve changes to processes or training for incident responders.

In practice, teams design a range of solutions to handle specific failure classes and reduce the frequency and duration of outages. Resilience improvements might be:

Manual

: Reactive activities performed by people (or AI agents) for novel, complex, or infrequent failures or threats.

Automated

: Reactive behaviors that are built into the system, correctly configured, and activated automatically to handle *recurring* failures that warrant investment in automation.

Architectural

: Proactive changes that reduce or eliminate the need for reactive actions (automated or manual) by addressing the underlying issue.

User Expectations

When making design decisions, remember that your system's users (e.g., clients of servers, not only the end users of those clients) have reliability and resilience expectations, and they also want to easily understand and manage the actions the system expects of its users.[1] This means that *you* should understand those expectations and plan accordingly, weighing what's required to meet them. Achieving this might necessitate cross-organizational work involving engineering, product, and sales teams. Ideally, you become the voice of the user, advocating for sound trade-offs within those teams and with leadership. This means you'll need to be informed about the risks, which teams or technologies can help address them, how much to invest, and when to stop.

With so much potential complexity to consider and manage, a principled approach is needed.

Google's Principles for Reliability and Resilience

Google SRE has formalized key lessons learned (based on experience gained over the years) into a set of principles supporting reliability and resilience. These principles out-

line a high-level philosophy and provide essential definitions of what is expected from reliable and resilient systems.

While the principles for *reliability* aim to provide reliable systems by assessing failures and controlling their impact on expected interactions, the principles for *resilience* aim to ensure reliable operation and recovery during failures outside the system's design parameters. Think of the adaptability of on-call teams or compute resources to address capacity shortages during outages, the effectiveness and continuity of recovery actions, preventing or limiting the duration of outages across multiple services, and controlling the risk of catastrophic loss to ensure business survivability.

The principles cover:

Risk management

: Having actionable data that provides situational and historical awareness about the health of systems and teams, and their capacity to handle failures and meet user demand

Change management

: Balancing the velocity of system changes against their size to maintain control over system complexity and minimize potential disruption to users

Dependency management

: Building system representations to help understand and control how systems depend on each other and work around failures

Failure domains

: Deploying systems into independent failure domains to reduce the risk of global outages and enable recovery after failures affecting one or more domains

Overload management

: Enabling systems to preserve the most valuable functionality during temporary or sustained excessive traffic and recover fully afterward

Input validation

: Hardening how systems accept data from external sources

Data resilience

Protecting systems from corruption or loss of critical data, preventing system failures, and safeguarding users from data loss

Qualification

Expecting systems to validate their resilience properties under realistic conditions and in appropriate environments

Emergency mode

Ensuring systems and teams can maintain operational control and recover from failures during outages of core services and infrastructure

We defined these principles because we wanted to focus our organization's engineering energy. Through Google's standardized production platform, compliance with many of the principles works by default. As a result, we were able to deprecate an ever-growing number of bespoke solutions and tools.

A principles-based approach like Google's can work for organizations of various sizes. You are in the best position to determine which risks matter most to your internal and external users, customers, or business partners. Use that understanding to influence your take on the principles.

Defining your own set of principles (and supporting guidance) for resilience and reliability will enable you to focus more of your engineering effort on categorizing common risks, prioritizing them by impact, and selecting actionable remediation strategies. The following sections will help you understand how to apply them.

Enforcement Through Technology

Because resilience is everyone's responsibility, these principles should be incorporated into the design of a system and throughout its lifecycle. The text of a policy or a standard will get you only so far. The resulting resilience should be made testable and validated wherever practical (see Chapter 14).

You might be asking yourself, how do we ensure conformance to these principles? A simplistic answer is to define a rule to follow (a policy), then provide supporting instructions for how to do that (a standard). This kind of documentation is important, but it's insufficient. The amount of awareness required to build a compliant service is likely to keep increasing. The constant evolution of technology and regulations, combined with shifting and conflicting goals and constraints, can contribute to cognitive overload for your engineers.

Technological solutions can help keep resilience expectations—and the decisions made to resolve conflicts—organized and applied by default (where practical), ideally without a human in the loop. Adding measurements, platform defaults, and programmatic enforcement to support your policies helps make your resilience goals more tractable.

At a very high level, you should:

Understand the risks.

Understand your reliability targets and error budgets, and assess how investments in resilience might affect your ability to stay within those limits. Consider what may help you avoid reaching those limits, and what solutions might be needed if they are exceeded.

Use common solutions.

Prioritize migrating to common solutions provided by your organization that offer desirable functionality out of the box.

Seek exceptions when common solutions aren't sufficient.

If common solutions don't support your system's or team's needs, seek exceptions from the standards and policy owners to implement a custom solution that meets their expectations. This covers impactful edge cases or novel situations where the better approach is a new approach or a variation of an existing solution.

Escalate remaining unacceptable risks.

If the cost of a solution substantially exceeds its potential rewards, escalate the issue to leadership so they can decide whether to accept the risk or prioritize additional engineering investment.

When you can provide easy-to-use infrastructure and building blocks that seamlessly enforce the underlying principles, you get something much more effective and efficient, with benefits that are:

Durable

You're probably familiar with the "superpower" of platforms—effective platforms hide technical complexity, offering users a smaller set of useful and safe controls. Even without a platform, defining and providing standalone solutions for the principles you've prioritized will improve the velocity and cost of achieving and maintaining conformance. To ensure durability, you must continue measuring the effectiveness of those solutions (e.g., through metrics such as usage, time to com-

pletion, and rates of complaints or praise) as feedback signals for ongoing investment. As your base of standalone solutions grows, you may eventually choose to build your own platform to host your reliability, resilience, and security practices.

Compounding

An organization's production practices naturally evolve over time. When those practices are codified in technology, changes are less costly and less disruptive. Complete components or layers of the technical stack can be replaced safely, with excellent cost-benefit ratios. Contrast that with changing policies or instructions and waiting for people to notice, adjust their thinking, and go through a process of trial and error to reach a new steady state. Such changes are slower and reduce the cognitive capacity engineers have available for top-of-mind priorities—or, more realistically, the top-of-mind priorities perpetually delay your ability to effect policy changes.

The concepts defined here are critical to keeping resilience agile. Threats emerge quickly, and their impact may be either tolerable or not deferrable. Therefore, the organizational capability to make changes fast (both proactive and reactive) is also part of being resilient. Using technology can effectively build that flexibility into your organization's resilience norms and practices. Implementing resilience expectations as code or procedures makes resilience durable, discoverable, and reusable for greater effect. Changes to these implementations can be rolled out quickly, while ensuring everyone learns about updates to familiar procedures.

The Role of Uniformity

Solutions for resilience should be simple wherever possible. Complex plans or methods might work, but resilience *must* work. Simpler systems and interactions are easier to understand, build, verify, debug, and maintain.

Uniformity can help reduce complexity. Increased uniformity of tooling, standards, and policies across production reduces cognitive load and improves cost efficiencies. However, forcing standardization could undo those savings if it ignores the nuanced requirements of different applications. It's important to recognize when a specific application requirement merits a bespoke solution, as well as to allow developer teams to compensate in bespoke ways.

It takes intent to find and maintain the right level of uniformity. A combination of empathy and rigor is needed to eliminate unnecessary complexity without making developers' lives unnecessarily hard. We recommend ongoing discussions on how the needs of differ-

ent organizational units align, how organizational units converge on practices and tooling choices locally, and how lessons learned are shared globally.

A related consideration is the difference in uniformity between new (greenfield) and existing (brownfield) systems. With new systems, you have the opportunity to design and enable all applicable solutions before they go into production. Existing systems, however, must be changed carefully to adopt those same solutions, taking into account each system's unique requirements, as well as the risks and costs of modifying production systems. With this in mind, you'll need to choose your focus carefully. Making solutions easier to adopt in greenfield systems will limit the expansion of the number of brownfield systems that don't have them. You can then focus on implementing solutions in the most critical brownfield systems first. The resulting simpler, more uniform production environment will deliver greater resilience at lower cost while helping your technical ecosystem become more reliable by default.

Risk Management

Risk management can directly influence technical investment strategies. Clearly articulated risks help an organization determine which risks to prioritize, and teams and leadership play distinct roles in enabling effective and accurate comparisons between them. One key goal is assessing the probability and impact of risks, although this can be nonscientific and subjective. Another is assessing the feasibility and cost of robust and durable remediations.

To help make risk decisions actionable, system designs should enable the collection of data about failures that have already happened or could happen. This data should be gathered across all systems and all layers of the production stack. Each part of the organization should contribute what it observes, using a common framework for expressing risk attributes. For example, methods such as STPA and CAST (discussed in Chapter 12) and Disaster Resilience Testing (discussed in "Disaster Recovery and Resilience") provide structured ways for systems and teams to identify risks

Armed with this data, the organization can decide whether to accept a given risk or invest in appropriate changes to architecture, automation, or operational responses.

The Role of Teams

Teams need actionable data to make informed decisions on risks within the technical stack of the systems they develop or maintain. This is the *vertical* view of the ecosystem

risks. Risk-finding methodologies such as STPA and CAST uncover risks and shine light on patterns and opportunities, but that's not *data*. Data is what lets us quantify our reliability and resilience posture or confirm risk hypotheses. It can serve as further input for those methodologies.

For example, teams are expected to record the impact and contributing factors of outages and act on this data to ensure the expectations of end users, customers, or business partners are met.

For consistent results:

- *Reactively*, teams should track resolution of contributing factors of outages, known architectural weaknesses, rate of change, efficiency of alerting and supervision, and system degradation before the degradation becomes user-visible.
- *Proactively*, teams should focus on the criticality and cross-dependencies of components, system degradation from which full recovery may not be possible, and anything else that can lead to products being unable to restart after failure.

This data enables teams to adjust engineering investment based on cost versus risk, test coverage and results, demonstrated track record, and remaining error budgets, helping reduce or eliminate repeat outages and avoid or shorten novel outages.

Teams are expected to take end-to-end responsibility for the resilience of all interactions by their systems. This includes how failures affect clients (users or other systems), operations, preparedness for recovery, and failure containment, in addition to the factors mentioned previously. Assessments must be proactive, especially for lower-probability, higher-impact failures with little or no observed history.

If data shows that interactions between systems cause outages, the teams that own those systems must cooperate. Organizational leadership is responsible for identifying and addressing gaps in ownership and the bystander effect (when no one responds because everyone assumes someone else will).

The Role of Leadership

The data from individual teams can be recorded and aggregated across entire organizational areas, to enable broader analysis. As a result, standards and guidance can be developed (think back to uniformity), allowing teams to prioritize reliability and resilience issues using central guidance so their systems don't lag behind minimum standards. The

goal is to shape the local optimizations teams make to improve overall reliability and resilience. This is the *horizontal* view of the ecosystem risks.

One example of central guidance is that observability data should be collected before, during, and after failures. The combined insights from this data cover the full failure lifecycle (from good state to bad state and back again), supporting analysis and helping meet the observability objectives described in "Observability and Monitoring". This matters in production and during development. For example, you can use failure injection to observe system behavior both when it is introduced and when the failure is removed, after varying durations (see "Fault Injection Testing"). The first part targets response to the failure, while the second part targets recovery after the failure—is recovery automated or manual, partial or complete, slow or fast? In addition, how closely does observability match the changes in behavior? It also allows you to observe whether the behavior is stable or varies with time (perhaps unexpected side effects kick in).

When evaluating risk and performance horizontally across the organization, leadership should pose the same risk questions to all critical product, system, and development/ops teams. This can be done through any suitable means, such as making data queries, building novel observability data, or simply talking to individuals (e.g., technical leads or advisory boards). The answers, representing *new and actionable* data on reliability or resilience, can then become part of the organization's aggregate knowledge (e.g., reports) presented to the relevant stakeholders.

Probabilities

Accuracy in risk assessment can be quite challenging. The classic risk formula—multiplying likelihood by impact—doesn't provide useful values when either probability or impact is very low. This is fine for low-impact failures, because mishandling them has a natural upper limit on consequences. But resilience is more concerned with the low-probability failures with high impact, because these failures can have consequences that affect business survivability (and most organizations target maximum survivability). Therefore, failure probability guides the design for resilience. This is the *cost-based* view of ecosystem risks.

For high- and medium-probability failures, regardless of impact, we recommend developing and handling scenarios that are low cost to execute and maintain. These mitigations should be enabled by default or be trivial and safe to actuate manually. Start with failures that can be resolved easily, completely, and across the largest possible portion of your infrastructure. Self-healing practices are preferred, so that issues are resolved more quickly and engineers get paged less. Server throttling, rate limiting, and rollout

blocks are examples of mechanisms that contain (limit) failures. Continuous probing of required behaviors against a server instance where all optional dependencies are disabled (via failure injection) is one way to validate resilient behaviors.

For lower-probability failures with *low* impact, explicitly accept the risk—defer all work, but keep the items in your backlogs or risk registers.[2]

For lower-probability failures with *high* impact, we recommend a different strategy. First, prepare a plan for how you would handle such failures today. If you aren't satisfied with the predicted outcome or the robustness of that plan, you have work to do. For each failure's plan, estimate the costs of implementing a reliable solution. Then, you're ready to define the order in which engineering should churn through them, and how many months or years it will take. Don't try to put engineering solutions in place to remediate all risks right away; instead, focus on addressing a few risks fully first, then move on to the next-highest-value risks. Here's a rough guideline to prioritize the risks, from highest to lowest:

1. Your system has already experienced the failure (e.g., accidentally deleting data).
2. Your system hasn't experienced the failure, but others have (e.g., unrecoverable backups).
3. No one in your organization has experienced the failure, but most recognize its realism (e.g., in-flight data corruption).
4. All the rest (known unknowns).

All scenarios (plans and behaviors) need regular testing. For more information on appropriate methods, refer to Chapter 14.

In summary, risk management should "shift left" the key decisions needed to deliver the appropriate amount of resilience, moving them toward the earlier stages of the software development lifecycle. This can be done by using a combination of data about risks and performance in the face of failures, fed through distributed teams to central leadership, and prioritized by probability, impact, and engineering feasibility.

Change Management

While necessary, technology changes are a continuous source of risk. The larger the scope of a change, the larger the area potentially affected, necessitating greater safety considerations before, during, and after the change. In addition, changes compound across all systems, teams, and workflows involved in production and operations.

Google avoids changing too many things in production too quickly. This includes making a single update to too many components and making multiple updates to the same component. The core issue is the tension between the need for development velocity and the need for understandable, manageable systems. As systems are becoming more complex and evolving more rapidly, especially in the AI era, the following points are becoming increasingly evident:

Velocity is crucial.

Rapid development is essential to keep pace with the demands of the business and to respond to failures.

Small changes are necessary.

Changes must be small and manageable to allow for easier reasoning about the system's behavior, especially when the complexity of interactions between systems grows superlinearly.

Balance is key.

If changes are too small, the rate of development slows. If changes are too large, the impact on systems becomes too complex to reason about, leading to reliability issues.

Google's change management policies and standards help engineers understand and manage the risks inherent to changes of any scale. In this section, we'll cover some practices that build resilience to changes throughout a system's lifecycle.

Progressive Rollout

Changes of all types (e.g., to binaries, data, or configuration, or when running experiments) should be made with progressively increasing scope and at rates that allow for failure detection in the earliest rollout phases. Detection must occur early enough to mitigate impact and establish confidence that SLOs won't be affected in the subsequent rollout phases. A simple goal here is to identify unexpected behaviors before they can affect a large proportion of serving capacity and burn an uncomfortable portion of the error budget.

Changes to data used across multiple failure domains contain a special risk. This category of risk includes global data, where there's the expectation of consistency in every failure domain, and therefore changes must be made in multiple places at once. Global data must be specifically identified, and progressive rollouts shouldn't be skipped for either

the data itself or the systems serving it. For example, consider using versioning for database schema and metadata changes, with activation by fast data pushes.

When dealing with globally consistent *bad* data, the probability of failures increases in inverse proportion to the time allocated for change supervision (covered in "Change Supervision"). To compensate for this probability, clients of global data should improve their resilience when handling both expected and unexpected bad data, in a fail-open or fail-closed mode as appropriate for the use case. We cover this in "Input Validation".

Change Concurrency

Avoid making multiple changes in a single rollout until they have been proven safe individually. For example, updating the binary and the data used by a system are better done independently, as otherwise it might not be clear which change caused the system to behave unexpectedly. Also, don't start to update your system in multiple zones at the same time; instead, update only one zone in the first stage of your progressive rollout. Wait for subsequent stages to update two zones at the same time, then more than two.

Any time we change multiple behaviors in a system at the same time, we increase the time, cost, and risk of undoing those changes if necessary. When the effects and side effects of multiple changes interact, failures may take more time to materialize, compound (making the eventual outage worse), and be harder to trace back to the changes responsible for them.

In complex environments like Google's, changes can be happening in multiple layers of the production stack at the same time. When it's not possible to serialize all changes, and you're forced to parallelize changes during a time window, you should avoid changing components that interact with each other. For example, while you are updating an application's server code, avoid updating the application's clients or database schema. However, it might be okay to update the network firmware or OS packages of that application's node. Always serialize changes to the interactions between the application's client code and server code.

Change Supervision

Changes should be supervised (e.g., using automated guardrails) to detect unexpected behaviors introduced to the system, using end-to-end observability signals as close as practical to the system's end users. Google's Canary Analysis Service (CAS) can automatically monitor the health of each replica undergoing a change and wait to mark the updated version as live until multiple replicas (canaries) demonstrate the change is safe.

For example, CAS will use observability metrics, many of which are already engineered into Google's production platforms, for CPU/memory usage growth, errors/exceptions, process crashes, prober results, etc. A simpler solution is having monitoring alerts reference a playbook describing how to pause the rollout, investigate whether alerts are related to the change, and resume or roll back.

You might want stronger supervision for systems in the highest criticality tier, such as infrastructure providers or any systems that are required dependencies for many other systems. It's good practice to understand and observe the health metrics of higher-level clients (clients of the dependents of the changing system). Practically, even though your system looks healthy during a rollout, failures in multiple systems that interact with yours can give sufficient justification to pause and investigate. To codify this supervision, you could include health signals from the dependents as a signal of your system's health.

Safe Mitigations

Teams should know what changed in their system and be able to safely revert those changes, using generic or specific mitigations. Some generic mitigations are:

Drain

Move traffic to other places not showing signs of failure.

Roll back

Revert your system to a known-good state. Most commonly, this is your binary (e.g. , if there's a bug in the code) or your data (e.g., if pipelines built new data incorrectly).

Upsize

Manage overload or higher serving costs by adding more capacity (e.g., replicas, quotas). This can be expensive and temporary, to buy time for more permanent solutions to scale the system.

Degrade

Manage overload or other specific conditions by reducing functionality to stay up rather than crash.

Block

If you encounter a new "query of death" that consumes excessive resources, or any other client behavior that affects the entire system, have a mechanism to

block it.

Using a rollback to revert your system can be a preferable safe mitigation (for binaries, configuration, and data), because you have higher confidence that the previous version will still work (although that's not always true, for example if the new logic has already changed data in a way that's incompatible with the old logic). If a rollback isn't helping (e.g., because the old state can't be made valid anymore), being prepared to safely complete a roll forward is prudent, too.

You should establish processes to ensure that relevant mitigations are available when needed, routinely tested to work correctly (i.e., make the expected change without unintended side effects), and practiced to ensure people know how to use them and feel comfortable activating them. For example, you may think you can do a rollback but discover during an outage that you can't. Practice and validate ahead of time.

Finally, mitigations should still comply with all change management principles: progressive rollouts, concurrency, and supervision. If your system state was changed such that the rollback only creates a new broken state, help yourself catch that early.

Risk Tolerance

There may be times when your system or organization has unusually low risk tolerance. Changes during these sensitive times require additional scrutiny. Before an important event, you might want to reduce the rate of changes or increase supervision to lessen the probability of an outage. Strategies might include more change progressiveness, less change concurrency, and postponing changes. Which systems are affected, for how long, and to what extent all depend on the situation.

Due to heightened sensitivity and the potential for outages, Google has defined and regularly activates operational periods that allow only minimal production changes. During these periods, nonessential code deployments to production systems are paused, to ensure stability and reliability. Several intermediate levels have also been defined to accommodate less sensitive and more routine production situations. Here's a brief summary of the levels:

Normal sensitivity

: Proceed with feature development and rollouts according to standard production practices.

Medium sensitivity

Proceed as usual with routine builds and rollouts, but postpone changes deemed high risk by the team.

High sensitivity

Reduce rollouts to only change types considered low-scope or high-value: break fixes, routine quota changes, location-isolated infrastructure turnups, or critical security fixes.

Highest sensitivity

Pause all elective production changes and any in-flight rollouts; only break fixes and critical security fixes may continue.

You could develop and follow a similar practice to apply, for example, during high-traffic holiday periods when staff levels are lower, or when your company is preparing for an important launch and wants to minimize the possibility of disruptions.

Defense in Depth

As a general principle, problems should be detected as early as possible throughout the software development lifecycle and during change management. Ideally, you would want all changes to production, manual and automated, to be implemented so that they can be proactively blocked when unforeseen extraordinary circumstances warrant it. Remember that for any breaking change that is caught by tests and blocked from rolling out, or detected by the application during a data push, your application shows resilience by avoiding the need for incident responders to apply mitigations. We can offer two examples of Google's approaches: feature flags and emergency stops.

A *feature flag* is a coding practice where changes with potentially high impact are fully implemented but disabled until a configuration flag is changed from the default "disabled" to "enabled." This can be used when introducing incompatible behaviors or changes to global data or systems that generate such data. This solution makes the rollback a matter of a simple configuration change, without concurrent unrelated changes. Feature flags generally have fewer dependencies than a binary rollback: they don't require fetching binaries, modifying the filesystem, or rebuilding containers. If disabling a behavior is safe, using feature flags can shorten outages. For maximum value, developers should consistently follow the guidance for adequately implementing them (e.g., including feature flags for all current features and maintaining flag hygiene, so flags are easy to reason about and use safely).

An *emergency stop* tells the system to stop its activities or to change its behaviors, in some predefined and inherently safe ways. That safety is necessary for reliability, but it also plays into psychological safety when deciding to activate a mitigation. For example, Google has a system that updates the kernel and firmware on machines. It's possible to pause changes to some or all machines if we're seeing unexpected behaviors and don't yet understand the contributing factors. Because activating this emergency stop can be highly visible, people need confidence that it won't introduce new failures and that the rollout can be resumed if another more critical migration is being delayed.

In summary, a set of independent systems should monitor the overall health of a system's software stack and its runtime environment, slowing down or stopping ongoing changes if disruptions are detected. Signals from the system's clients can provide situational awareness that helps assess the health of systems undergoing change. If you can alert when more outages are being declared compared to a recent baseline, teams can be proactively prompted to look for contributing factors outside their systems, making incident response more effective.

Dependency Management

Dependency management is about how your system depends on other systems and how you enable other systems to safely depend on yours. This also applies at the level of components and assets that make up a system.

Teams should actively manage their dependencies and avoid dependencies on systems that do not have appropriate reliability targets to support the system's reliability needs. When you understand how system parts (e.g., RPC-based microservices, databases) and supply chains (e.g., software libraries, firmware, hardware) affect the overall functions expected by your users, you can more effectively apply engineering effort and manage risks. And when you regularly review dependencies, you can actively respond to changes—for example, you might notice when a small piece of business logic becomes critical but is found to be operating without sufficient reliability for such a dependency.

The more layers in a function's dependency tree, the more opportunities there are for a single failure to have unintended consequences. Dependency loops (cyclic dependencies) or fragile dependencies can result in a risk of deadlocks or cascading failures at runtime, which could lead to local or global outages. Supply chain failures (external dependencies) can disrupt build pipelines, which could delay outage recovery or business growth. Designing for resilience needs to account for these possibilities.

System developers and operators should be able to:

- *Describe the composition* of the system, so that everyone can have a consistent understanding of how it's put together and the nature of its dependencies.
- *Document the dependencies* of the system, so that these relationships can be monitored (and optionally enforced), both statically and in real time, according to criticality.
- *Define usage scenarios* for the system, so that its users can understand the safe and predictable ways of depending on it.
- *Declare failure scenarios* handled or not handled by the system, so that its users can make informed decisions about additional resilience they might need to build.
- *Understand the users' use cases* indirectly supported by the system, so that this signal can influence the prioritization of risk burndown and any relevant compliance efforts.

Combined, these capabilities will help ensure that your organization can analyze, monitor, audit, and enforce interdependencies between systems, to identify and manage resilience risks.

Representations and Controls

Useful dependency management includes processes and tools for two complementary outcomes: representing your systems correctly and having controls for maintaining expectations for the dependencies.

Representing your system correctly is the first milestone. You need to account for the full set of production assets that belong to the system, categorized into components and annotated with metadata about environment and type. Production assets include application processes, databases, nodes, and any other resources that allow the system to be built, started, and run in production. Include the most important details for each, and increase granularity where it makes sense.

Dependency management controls are your next milestone. First, you need mechanisms to lock down which dependencies a component is allowed—for example, by checking compiled-in libraries or allowed RPC endpoints against defined ACLs at build, configuration, startup, or runtime, as appropriate. Second, set up alerts for violations, which notify you if unwanted dependencies are introduced. Finally, ensure that your system representation remains up-to-date, and make changes to your system definition to correct missing or wrong data.

MODELING A DEPENDENCY

To model dependencies, it can be helpful to define the essential attributes of a system and its dependency relationships.

In many cases, a system representation can be generated automatically with a reasonable degree of accuracy. To create a more accurate representation, you can provide a system definition: a configuration file that specifies how the technical assets making up a system should be categorized when constructing the system representation.

A *system definition* can include the following concepts:

System

Represents a set of production assets grouped together based on their ownership, implementation, functionality, and responsibilities.

Component

Groups production assets that provide the same functionality. Those assets might operate in different regions, release stages, or other organizational units.

Component type

Describes the core purpose of the component (e.g., serving user traffic, performing batch processing, monitoring, or executing operational changes).

Environment

Represents the context in which production assets fulfill business goals and enables consistent but isolated system deployments across environments. Components in "prod" affect users more directly than those in "staging," but "staging" versions of systems must still behave similarly to the "prod" versions. Importantly, production systems should not depend on non-production assets.

A *dependency relationship* can include the following attributes:

Mechanism

How the dependency is technically realized. This can take the form of:

- *Configuration*: When system operating parameters change in response to factors external to the system (e.g., configuration or data pushes define which addresses and timeout values the system uses for backends or caches)

- *Execution*: When parts of the system logic are provided and configured by a component in a separate execution environment (e.g., via RPC) or technology stack layer (e.g., via operating system modules)
- *Physical*: When systems need tangible resources, such as machines and the power and cooling required for datacenter operations

Criticality

The consequences to the system if the dependency fails. How much to prepare for a dependency's failure is guided by the criticality of the functionality that dependency provides. Failures in the functionality required for a critical user journey can have impacts that users care strongly about. Dependencies can be classified as:

- *Optional*: When failures can be tolerated without loss of critical functionality. The system is fault-tolerant to this dependency and should fail open when appropriate.
- *Required*: When failures disrupt critical functionality. The system is not fault-tolerant to this dependency and might fail closed or fail secure when a sufficient fallback isn't available to substitute for it.

Conditions

The stage of the system lifecycle during which the dependency is required. The stages include:

- *Build*: Required to build the system (e.g., compile or install it)
- *Startup*: Required for the system to begin serving the functionality users expect
- *Runtime*: Required for the running system to perform its expected behavior

---

Resilience to Dependency Failures

To improve resilience to unexpected behavior in dependencies, focus first on broad categories of failure, then expand to the failure categories that occur most frequently. When analyzing failure scenarios, focus on direct dependencies but also consider indirect ones.

Regularly demonstrate that optional dependencies are indeed optional. If you inject failures into every dependency marked as optional (fault-tolerant) in your system representation, your system's observability should not report new failures in any critical func-

tions. If you get alerts, find and address the gaps. If you can't confirm a dependency is optional, it should be considered a required dependency.

For required dependencies, plan what you expect to happen during failures, and regularly demonstrate that those plans work. When you inject failures for one required dependency, the alerts you get should be for the specific functions and nothing else. Also, those alerts should automatically stop after you stop injecting failures, indicating that the system recovers on its own. If what you observe in practice differs, find and address the gaps. For more on this, see "Testing Dependencies".

When investigating and validating the fault tolerance of dependencies, you may use any failure injection mechanism appropriate for the specific production asset type. Use systems in a non-production environment, such as "staging" or "dev," to observe how the system reacts, and improve it until it matches the expected behaviors. Consider injecting occasional failures into production as well, to confirm that the expected resilience is maintained while staying within acceptable error budgets.

Defense of Dependents

The inverse of a *dependency* is a *dependent*—something that depends on your system. In this relationship, your system is another system's dependency.

You should consider how changes to (and failures in) your system affect its dependents, especially mission-critical ones. Beyond direct dependents, also think about effects on the broader ecosystem surrounding your system: business uses, operator habits, operations playbooks, cross-system interactions, and recovery procedures.

Imagine that an outage puts multiple systems in a degraded state because their required dependency failed. The incident responders must decide how to allocate their efforts and limited time across all affected systems.

If your dependency management platform helps discover a failed system's *dependents* and their criticality, CUJs, and locations:

- *Incident responders* can identify and prioritize restoring the functionality that the critical dependents require. The longer the recovery takes—from minutes to hours—the more this matters.
- *Planners of maintenance* or other disruptive changes ("controlled outages"; see "Implicit SLOs") can help the affected dependents prepare for whatever might po-

tentially go wrong or can start changes first in the locations with the least critical dependents.
- *Engineers building automation* can determine where outages will have the greatest impact on dependents and prioritize safe automated failure handling—on the order of seconds—for those CUJs first.

If your change management system helps systems declare and make discoverable their current risk tolerance, then it becomes possible for your dependency management system to find systems affected by an outage and ask them to adopt (or enforce) a higher level of change scrutiny. Incident responders or automated maintenance processes can then take action to:

Protect remaining dependencies

Whenever your system enters a degraded mode, you might want to protect its remaining functionality by asking healthy dependencies to pause changes, for example by setting their risk tolerance to "highest sensitivity" (see "Risk Tolerance"). This can be especially valuable for mission-critical systems or while rolling out organization-wide critical changes, to minimize risk from simultaneous lower-priority changes.

Protect fallback capabilities

Similarly, you might want to lower the risk tolerance of your system's fallback mechanisms (see "Managing Fallbacks") or operational capabilities (such as incident response and recovery systems). This helps avoid unrelated new outages affecting your already degraded system's recovery.

Managing Fallbacks

Teams should assess the availability of alternatives (fallbacks) for required dependencies, whenever failure of those dependencies could violate SLOs beyond acceptable margins. Outage duration can often be reduced if you can "switch" the affected system to an alternative version of a damaged or defective dependency.

For example, you could build and maintain an earlier version of your system or a completely distinct implementation with a low-dependency architecture and/or fewer features. Google uses both approaches sparingly for situations where the main system's functionality is especially valuable to justify the inevitable added costs. Each has its strengths and weaknesses:

- It's easy to keep earlier versions (just use older releases) running in a separate environment or failure domain (covered in the next section). The downside is that the revisions will share many parts with the main system, and failures in those parts aren't covered by this "fallback."
- It's much more costly to build a full alternative, because it needs to stay compatible or interchangeable with the existing implementation. The upside is you can create a fully distinct stack with little shared fate and thus very few failure types capable of disabling the primary system and the fallback.

To support this, systems with many dependents should take responsibility for providing their own fallbacks (e.g., earlier versions, low-dependency implementations, etc). The need to provide these alternatives varies with the criticality of the system—for example, systems that are critical to managing production warrant stronger fallback options. For optional dependencies, the cost of managing fallbacks might not be worth it.

Failure Domains

Splitting your service into multiple independent failure domains reduces the risk of broader and global outages of that service, but only if done correctly. Otherwise, you might have only an illusion of safety. If your failure domains are not fully independent, a single failure domain being down can lead to outages in others, potentially taking down the entire system.

To take full advantage of redundancies designed into your system, it's important to limit how many service instances could be disrupted by a single failure (or by a set of related failures). This requires assessing the application's serving path (from clients to backends), the internal dependencies (interactions among components and with upstream services), and what failure domains the upstream infrastructure can guarantee (e.g., cloud regions as units of logical and/or physical isolation).

When considering failures, distinguish what you control versus what your infrastructure promises. Once you've determined which failures you must handle, plan for the occasions when the infrastructure might fail to deliver on *its* promises. As discussed earlier, you will plan and prepare differently for higher-probability and lower-probability failures.

Your goal is to ensure that failures in one domain (e.g., cluster, partition, region) don't disrupt other domains beyond what can be managed practically and cost-efficiently. Determine where isolation and colocation are most critical, and aim for an architecture

and a deployment strategy that enable the application to be subdivided into self-sufficient parts. Each part should contain all the dependencies it needs (with the exception of allowed global upstream systems or data) and be able to grow its capacity or be repaired independently of other parts, to address local bottlenecks due to usage changes or failures. The simplest strategy is for the entire stack to run locally, but even then you have several options to consider and optimize for.

Type of Failures and Domains

How to break up a system into failure domains often depends on the nature of the expected failures. Broadly speaking, application failures can originate due to:

- *Client activities* (e.g., a "query of death")
- *Application component* behaviors and interactions (e.g., a required dependency starting to crash after a recent rollout or data push)
- *Upstream infrastructure* outages (e.g., an outage of a cloud provider's APIs)
- *Physical infrastructure* outages (e.g., a datacenter cooling outage causing multiple machines in a zone to be shut down)

The nature of the failure often determines what failure scopes are applicable, which in turn helps you make "isolation walls" within your system's architecture. At Google, we've found the following types of isolation helpful for developers when defining failure domains:

- *Logical geographic isolation*: Isolation of regions, zones, and nodes containing the system.
- *Logical functional isolation*: Isolation of system functions based on which clients are allowed to use them.
- *Infrastructure isolation*: Isolation across hardware racks and network infrastructure.
- *Operational isolation*: Isolation of operators and automation, with sufficient credentials and capabilities to change systems.

Logical geographic isolation is a common way of defining failure domains (e.g., zones and regions in cloud deployments). If global outages are not acceptable for a particular application, deploy it to multiple zones within a region, and to multiple regions. This provides you with protection from certain common physical failures, as well as software failures or rollouts within your individual systems. If you choose a conservative footprint, to reduce costs, prepare clear and regularly tested plans for quick turnup in a new region.

Logical functional isolation can be achieved by running application stacks dedicated to particular functionality (logical partitioning) or sets of users (user sharding). For example, certain functions or distinct populations of users may need different shapes of compute resources or customized compute containers, each of which could become its own failure domain.

Infrastructure isolation ensures that you don't end up with all servers running in a single rack or reachable via a single networking domain (whether the racks and networks are logical or physical). Your visibility into and control over these failure domains may be a feature of your available infrastructure, ideally enforced by default. If you experience such failures, you might need to develop your own mitigations.

Operational isolation focuses on avoiding loss of the operational capabilities required for the system to keep functioning and evolving to meet user demand, including normal updates or repairs after failures, while still following change management principles. For example, you need on-callers who understand the system well enough to change it quickly and safely, and those on-callers need appropriate playbooks and tools. Google typically partitions on-call teams into two shards that serve on-call at different times, and assigns two on-callers (primary and secondary) from each shard to the same system. As a result, the current on-caller has one local fallback person and two more from the remote fallback team, so the system is resilient to on-callers becoming unavailable during their shift. In addition, Google provides more than one way of making playbooks available to on-callers, in case the normal playbook server is affected by an outage. As a result, operational capabilities are also isolated from upstream infrastructure outages.

Decide which failure domain categories you want to prepare for first, and iterate to handle the rest. At the organizational level, define an appropriate failure domain taxonomy that considers your product requirements and business needs, and guides developers and SREs in what to prioritize and how to implement it.

Aligning Failure Domains

Aligning your system deployments to failure domains is the core of resilience design, ensuring that a single failure—of hardware, network, or coding logic—doesn't disrupt more of your system than you expect. Well-aligned deployments improve your organization's ability to operate, recover, and plan capacity.

When you align your system's architecture to the boundaries of physical and logical domains, no group of components should reside in only one failure domain if you aren't ready to lose that entire group at once. Check that each group spans multiple failure do-

mains for every applicable type of failure domain (e.g., region, zone, device, network fabric, etc.).

Consider Figure 17-1. Any given node will reside in some zone and region. If your system is distributed evenly across two regions, with two zones per region and nodes distributed evenly across those zones, the loss of a single zone will result in the loss of at most 25% of your global capacity. If instead you used only one zone per region and ignored anti-affinity measures,[3] you could risk losing 50% of your global capacity.

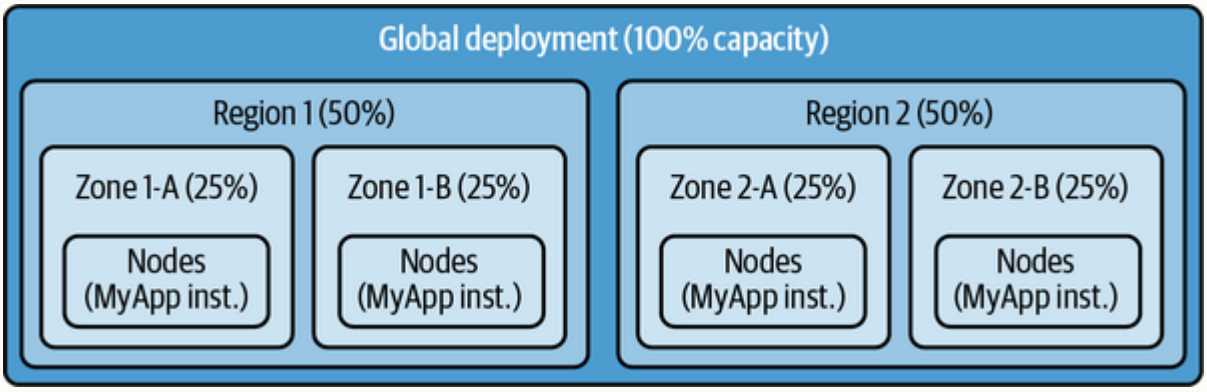


Figure 17-1. Global deployment example

You might not know if common dependencies exist in the lower layers of your system's production stack. From Figure 17-1, those nodes will also be located in a datacenter rack within a cluster assigned to some zone. The infrastructure teams (or cloud service providers) that own those layers should be aligning failure domains, too.

Another antipattern to avoid is distributing system components too widely. This may reduce resilience. If a system has two interacting components that reside in separate network fabrics, this may seem like a good idea (network diversity), but only if each network fabric has replicas of each component that are configured to prioritize interactions with their local counterparts. Otherwise, the moment either network fabric fails, the system loses that interaction.

Finally, you should manage the size of failure domain instances. For example, as traffic increases and your capacity needs expand to match, consider adding more zones (rather than growing the existing zones) to ensure the size of any given zone you might lose stays within your comfort level. This type of scaling strategy not only gives you more growth options but also lowers the risk of failure domains becoming "too large to fail" or causing cascading failures due to the thundering herd effect (when a large failure domain fails, its traffic can overwhelm smaller alternative failure domains, as well as the large domain itself when it restarts). Any time a larger failure domain fails, a generic mitigation such as draining that segment might stop being safe because of the resulting in-

crease in load on the remaining domains. In such cases, it may be better to manage the resulting side effects by limiting traffic shifts.

Recovering Failure Domains

We've already established the importance of containing small failures and not allowing them to turn into widespread outages. The minimal expectation for aligning failure domains is that all required dependencies are available within the local failure domain (regardless of how many remote failure domains also offer the same dependencies). This should be true for as many types of failure domains as possible, within the limits of cost effectiveness and reason for each specific system.

However, it's still important to design for resilience to multiple failure domains, or failures that take so long to mitigate that multiple failure domains fail. In other words, you need a strategy for when all failure domains fail. Incident responders can attempt to contain the spread by applying safe mitigations, activating broader mitigations, and enabling the "highest sensitivity" change mode. But when the spread of the failure is eventually stopped, some proportion of your capacity may remain disrupted or unusable. In some situations, a fix must be repeated across many failure domains.

Suppose your system has an RTO of 24 hours. (RTOs, along with RPOs and RWOs, are covered in "SLOs"). This means you must have a validated method of restoring service within 24 hours. If you can't close the capacity gap and restore the required functionality within that time frame, your promise is broken. If your system is critical, and users expect a near-zero RTO, this implies near-instantaneous failover to a healthy replica, which may require critical data to be written to multiple replicas in real time. If an outage is disrupting too many replicas, the key to preventing substantial SLO violations is recovering disrupted replicas to full functionality faster than the failure spreads.

To stop the spread of failures across multiple failure domains and recover your application, your operations teams need a small set of capabilities. Teams should know how to perform the following steps:

1. *Isolate* a system's affected failure domain. Stop the components in that failure domain from serving clients and influencing ("poisoning") unaffected failure domains by effectively removing them from those domains' decision making and execution paths.
2. *Repair* the system there. Fix the system inside the isolated failure domain, without relying on other domains. Using other failure domains may be acceptable for expediency, but if this is the only possible way, it increases survivability risks.

3. *Reassemble* the system from repaired isolated domains. This replenishes system capacity lost with failure domains and enables full recovery (survivability) if all failure domains were lost.
4. *Reach* into nonlocal failure domains. This capability enables repeating steps 1 through 3 as many times as needed, each time returning one failed domain into service, all from a single healthy failure domain (such as the "pristine" system copy we cover in "Preventing Loss").

The most expedient option is to "seed" the recovery from existing functioning replicas, but this option doesn't suffice if you seek maximum survivability, because it doesn't account for having no functioning replicas your application can reach (e.g., due to a network failure or a latent bug in global data). Prioritizing recovering a local failure domain in isolation from other parts of the system helps make it more resilient to large-scale loss, for the lower-probability but imaginable cases where all replicas have been disrupted.

Expediency also matters because systems in a broken or stranded state degrade your organization until they are returned to a usable state. Any components you put "in repair" are not getting regular updates, which includes security and compatibility changes. Keep in mind that when you bring components back online, you might have data consistency and logic consistency issues to resolve as part of reassembling the previously isolated failure domain.

Miscellaneous pitfalls to keep in mind:

- Recovery of nonlocal failure domains must include checks and balances to mitigate the risks of having one actor make changes across multiple (or even all) failure domains. This follows the change management principle of avoiding situations where fixing one thing breaks another.
- Recovery capabilities with inherently global powers must run only in locations selected for their safety. This helps reduce the risk of these capabilities becoming unavailable due to an outage or being compromised and abused. The principle is similar to using safes rather than laptops to store security keys used during fleet-wide recovery efforts, as safes are more secure.

Your application might also need to customize how it distributes traffic among failure domains. For geographic isolation, using load balancers is standard practice, for example based on client-to-server proximity. For functional isolation, load balancers need a way to differentiate which shards of users are served by which backend groups, which may require per-user naming and supporting logic in the application backends. For physical

failure domains, no action is needed, as the necessary infrastructure is usually part of domain design. You just select which machines in which racks to use, assuming the racks are interconnected so all machines can reach each other.

Preventing Loss

How failure domains can help prevent the loss of global state—a condition that can limit your ability to return systems to a correct and/or usable known state—deserves some attention. Global state is often unavoidable, and managing it requires special care because failures can easily cause global disruptions that are difficult or even impossible to repair.

An example of global state is globally replicated metadata used to find all the places where a system stores the distributed data it serves to users. It might be possible to rebuild such information given enough time, but recovery could take a very long time and success might not be certain. In some cases, you can avoid the need for rebuilding altogether by statically compiling and distributing the global data. Where practical, this is usually a better option.

As a general principle, all inherently global components and functions should be designed to preserve, within at least one failure domain, their pristine, known-good state (data, configuration, and binaries). While it's critical to back up data, that's not enough. You must also be able to restore your system's components to a functioning state (by correctly configuring and starting its binaries) so that the system can load the previously backed-up data.

Failure domains can help prevent loss by preserving copies of your system's global state, but in practice this is more challenging than you might expect. You could designate a small number of failure domains to hold the "pristine" fallback copies of your system, adding an extra layer of isolation to minimize propagation of failures that could damage or destroy those copies. Failures resulting from accidental or malicious errors in software logic, in real time or delayed ("time bombs"), could be reduced if changes happen in these failure domains last and the rate of change is substantially reduced. For example, a system that uses logical partitions (e.g., for user or traffic isolation) could create a "pristine" partition where rollouts are delayed by multiple versions or weeks, emergency rollouts are prohibited, and only monitoring traffic is allowed. Such failure domains can't necessarily serve as "warm standbys"; they are used not to serve emergency traffic but only to recover other failure domains using the preserved global state.

There are a few questions to consider in determining whether this is a worthy investment for your organization, and, if so, how to implement it. How many failure domains should

you designate to recover all functionality before risk of total loss becomes high? Is the data needed to recover the functionality available, and can it be used in that failure domain? Could security protections block recovery because the protections were designed to fail closed during disruptions?

Defense in depth can alleviate some of the risks, especially with regard to handling overload from expected and unexpected causes. We'll cover that next.

Overload Management

Load balancing across failure domains, combined with horizontal scaling or upsizing of failure domains, can go a long way toward avoiding overloading parts of your system. However, it won't address edge cases like when demand spikes too quickly (e.g., due to external factors or a query of death) or because it's impossible to shift demand elsewhere (e.g., due to traffic-specific factors or all other failure domains are already disrupted). You need a complementary capability to keep compute resources useful when overload is unavoidable.

Overload can become widely damaging when systems start to crash and stop handling traffic altogether. Systems receiving excessive traffic (whether total or per client, transient or sustained) should not fail in a way that's disproportionate to the overload or require manual intervention to recover once the overload ends. For example, a system provisioned for X but receiving 10X instead should continue to serve close to X instead of close to 0, and should return to serving X without manual intervention once the overload subsides.

A reduction in steady-state reliability is often unavoidable during overload, but that's an appropriate resilience trade-off. Broadly speaking, three approaches can be taken here:

Server choosing which requests to prioritize or ignore

Reducing the compute resources allocated to lower-priority traffic, even to zero, may be appropriate. This will result in lower-priority traffic receiving a high percentage of errors (up to 100%), even though it has "done nothing wrong."

Server providing lower-quality responses

Results could be served using stale cache entries or with fewer interactions with dependencies. This is a valid option if, for example, 80% of the response quality can be achieved with 20% of the computation.

Client reducing demand on the server

When the server starts to signal overload, clients can adjust their request rates, prioritizing the most useful, lowest-cost requests.

You should design your systems with a combination of autoscaling, load balancing, overload shedding, per-client or per-user soft and hard quotas, and so on, and regularly verify the effectiveness of overload management procedures. Client code should be designed to reduce demand when the server is overloaded, using client-side adaptive throttling, exponential backoff retry logic, and traffic labeling by cost and criticality. All of these reduce reliability loss during overload.

To test the system's resilience to overload, you can incrementally increase traffic until the observed goodput stabilizes. Goodput represents the effective throughput of a system, specifically measuring the work successfully completed to fulfill client requests (not including failed requests or retries). The stable goodput indicates the amount of useful work your system can reliably perform under any level of overload.

The overall goal is to continue performing useful work during overload. Success is determined by the extent to which the system is able to keep handling its highest-value operations. If that capability drops to zero, the system is effectively broken. A resilient system keeps it above zero for as long as possible and enables rapid recovery if it does drop to zero.

Avoiding Cascading Failures

Cascading failures are dangerous because they can easily lead to global outages. The "cascade," or "domino effect," occurs when a failure damages one part of the system, reducing its capacity and causing damage to other parts. This reduces capacity further, creating a positive feedback loop that soon spirals out of control.

Overload results in resource shortages. Shortages in CPU, memory, threads, file descriptors, or network bandwidth can manifest in various secondary effects that snowball into failures. Examples include:

Long queues of requests

Requests pile up as the CPU is unable to process the inflow, and the resources allocated to each request are not released.

Request deadline misses

As request latency increases, RPC deadlines expire, leading to repeat requests and wasted resources.

Container restarts

Containers can get restarted or evicted for exceeding allocated limits or not responding in time to health checks, including because of thread pool or file descriptor exhaustion.

Cache invalidation

Processes in the restarted containers have empty caches that must be repopulated, placing extra load on the computation and storage subsystems.

To avoid cascading failures caused by overload, you need mechanisms that can lower the amount of traffic to a level the system can bear without failing and enable recovery efforts for the already failed capacity.

Server-Side Controls

A system resilient to overload is designed and configured with two capabilities: to reject requests when overloaded and to fulfill the remaining requests sufficiently, even if partially. You might know these capabilities as load shedding and graceful degradation, respectively.

Load shedding avoids spending compute resources on some requests by rejecting them from the start, rather than waiting until overload conditions force them to fail. Any resources that would have been wasted partially processing those requests are instead made available to serve other requests fully.

Design expectations include:

- The system should be able to detect when it's overloaded, using a combination of objective indicators such as request queue lengths, memory, available threads, etc.
- The system should have traffic labeling for requests (e.g., RPC service and method names and parameters), indicating when it's okay to drop them (e.g., under moderate, severe, or extreme overload levels) based on cost and criticality.
- The system should return an error status clearly indicating the overload condition and whether the client should retry (e.g., "overloaded," which may trigger a retry, or "overloaded; don't retry").

Graceful degradation reduces the compute resources spent on accepted requests. By serving degraded responses, you provide some value instead of zero value, as is the case with requests rejected due to load shedding. Clients get responses that are less accurate or contain less data than normal but are cheaper to compute.

Design expectations include:

- The system should have clear degradation options for each supported request type, so that responses remain valid (e.g., how stale cache entries can be before serving them becomes unsafe, or what percentage of searched data can be skipped before returning partial search results).
- The system should add optional quality indicators to response metadata (such as different status codes for success), for use in resilience validation (whether expected degradation occurred) and to help clients discover and react to degraded service. For example, you might include cache staleness timestamps or the percentage of data or backend queries ignored when responding (as long as these indicators are not accidentally exposed to external end users).

Client-Side Controls

A resilient system enables its clients to proactively reduce demand on an overloaded server by using client-side *retry logic* and *adaptive throttling*. Such clients can react similarly when a server resumes after a full failure. (Note that this applies only to clients over which the system has influence, for example because they are part of the system's end-to-end architecture, and it assumes that the system or organization specifies a standard for client-side controls.)

Retry logic determines how a client proceeds when a request fails. If the status code indicates an ephemeral (retriable) error, the client might retry the request. Your design goal is to set expectations for *how* to retry and to enable all systems to easily follow that standard.

Design expectations include:

- The client should use exponential backoff, which reduces server load and the need for load shedding.
- The client should randomize the exponential backoff (introduce jitter), which minimizes load spikes from multiple clients affected by server load shedding within the same small time window. This may be especially relevant for infrastructure systems, which typically have many clients.

- The client should carefully retry when a large volume of work gets queued up while the server is unavailable, which avoids exacerbating the likely overload conditions during server restarts. These retries should be limited to a single level of clients, so that clients of clients don't create a cascade of retries.

Adaptive throttling dynamically adjusts client requests based on success rates and reduces traffic to avoid wasting server resources when the likelihood of errors is high. When a client observes that many recent requests have failed, it can choose to stop sending new requests (assuming they would fail, too) and apply retry logic only to the most important failed requests. This can substantially reduce server-side overload.

Design expectations include:

- The client should understand the error categories for requests and the likely recovery time for each, so that skipping the requests can be timed accordingly.
- The client should self-regulate by calculating the probability of getting an error based on the recent history of request failures in each category. As the number of failed requests increases, so does the calculated probability, causing the client to further reduce its load on the server.
- The client should consider criticality information (e.g., from the dependency management system's specification) when calculating that probability. For lower-criticality workflows, lower error thresholds may be sufficient.

Systems don't always control client code, so specifying behavioral guidelines (e.g., in published specifications) is key to informing potential clients. For clients that your system *can* control or influence, providing a default implementation via a client library is even better. Such a client library can be included in API gateways in front of HTTP-based services, or directly into user applications. At Google, we often use standardized client libraries that include client-side controls and integrate them with our production platform.

Isolating Overload

When your system is overloaded, it is ideal to isolate failures (error responses) only to the clients that are causing the overload, so other clients are unaffected.

The server-side and client-side controls covered in the previous sections can be applied on a per-user basis. If user isolation is valuable for your system, you may want to consider capacity provisioning with usage commitments and *per-user quotas* based on those commitments. You can then reuse these resource quotas when designing per-user bud-

gets and parameters controlling each specific overload mechanism (load shedding, graceful degradation, retry logic, adaptive throttling, etc.)

For a system that's entering overload, the goal is to react quickly whenever any per-user quota is exceeded, and even more quickly when the excessive traffic is directly linkable to overload symptoms. For example, during CPU overload, all per-user thresholds for requests with high CPU costs might be *adjusted proportionally* and become *strictly enforced*. As a result, each affected user will experience X% more load shedding and Y% fewer retries, proportional to that user's quota.

For a client interacting with an overloaded server, errors specific to overload should aim at informing the user of actionable changes in behavior that could help relieve the overloaded resources. Client retry budgets can be defined to control how long to keep retrying before informing the user of the overload (e.g., with a status message such as "pause sending more requests for now"). On the server side, errors specific to overload should aim at relieving the overloaded resources.

Design expectations include:

- The server should implement a per-request retry budget. After the same kind of request fails X times, propagate the failure to the client via an overload error because the system's backends are likely overloaded, which indicates a more durable condition within the system or its dependencies.
- The client should implement a per-service retry budget. If the retry ratio exceeds Y% across all requests the client sends to a server, propagate the failures to the user. This limits retry attempts across failure domains of the system, but only when many of those failure domains are overloaded.

In summary, even in a well-designed environment your users probably expect the occasional overload, and they may also expect to be informed if they can do anything about it, whether or not they contributed to the condition. Servers and clients should be designed to manage overload internally by detecting overload conditions and taking actions to reduce load where possible.

Actionable Mitigations

When the overload response configured for your system doesn't suffice and you are facing a cascading failure or another outage, some emergency actions can help:

Block sources of excessive demand

You can remove certain sources of traffic by denying them network access to the system. If the system is subject to queries of death and can identify and block those queries, this might be the first action to take.

Provision additional resources

Before too many fail domains or required dependencies are disrupted by overload, causing your system to collapse, try increasing resource quotas in the existing failure domains, adjusting load-balancing for better utilization across failure domains, or turning up new replicas of system components, until adding resources is no longer enough to stabilize the system.

Prevent server restarts

Disable any production automation that might be trying to reset components that have stopped responding to infrastructure health checks. Overloaded systems are more likely to exhibit those symptoms, but restarting them will definitely remove capacity.

Expedite server restarts

If you determine that overloaded servers are not producing goodput, restarting them in a controlled way may enable you to restore lost capacity.

You should also have a plan for a cold start of your system while users are impatiently trying to reload clients, as they check whether the service has recovered. A possible side effect of client restarts is the "thundering herd" problem mentioned in "Aligning Failure Domains". Clients sending too many retries can severely hamper recovery because the system might never achieve a good enough state to actually respond to incoming requests.

Input Validation

Data is another source of resilience adventures. Input to your system might come from sources with varying degrees of quality or security controls. The robustness principle in computing, which can be expressed as "be conservative in what you send, be liberal in what you accept," applies to how your system should handle all inputs. We might want users interacting with our systems to follow rules, but we should still expect accidental (or malicious) bad inputs. Systems should fail safe in a way that preserves functionality after encountering bad data.

Your system should expect and be prepared to handle bad data anywhere defects could have been introduced, including data originating outside the system, data provided by a dependency, and data read from storage. The diligence in handling input failures before they hurt your users or your missions will pay off.

Validation Outcomes

As engineers and organizational leaders, you want to minimize the risk of bad inputs leading to outages. Inputs such as configuration and code are controls; they can change execution logic and trigger various failures. Don't let them!

Design expectations include:

- Suspected bad configs and bad user content should be detected quickly, accurately, and automatically before causing significant impact. Engineering focuses on helping teams recognize sources of input risk and maintaining a set of recommended response strategies and implementations. Use applicable algorithms to detect all relevant types of bad input.
- It should be possible to mitigate outages caused by bad data within a defined and reasonable time (minutes, not hours) after determining the right corrective action. Engineering focuses on identifying and preparing corrective actions (safe or broad mitigations) for known and potential scenarios, such as implementing data rollbacks and revising allowlists or denylists, as well as on-caller responses for manual corrective actions. The goal is defense in depth for when bad data detection might fail or experience a regression.

Note that here we're focusing on resilience to changes during the system's execution, as opposed to changes during the software development lifecycle; those are covered in "Change Management".

Types of Bad Input

How your system reacts to bad data will depend on the nature of the input. Bad inputs include:

Empty updates

Replacing existing data with an empty update can amount to deletion. The consequences can be huge—for example, locking people out of systems when the ACL becomes empty, or servers crashing upon finding the set of critical backends

empty due to a file permissions error. Always check if the input is empty, and assess if that's a valid possibility.

Partial updates

Similar to empty updates, truncated updates can amount to partial deletion. If an error occurs during a streaming RPC, the system may know it has received only a partial update. But data truncated by the code that generated it can be harder to detect with precision and to diagnose later if it goes undetected. For example, how can the system confirm that the data push was meant to be 10% shorter that usual? Check for extra information that could validate data integrity.

Corrupted updates

For the smallest data elements (e.g., bytes or bits), error detection mechanisms such as checksums are required. These can be applied at every layer of data transfer and storage protocols (many do this by default). For example, integrity checks can detect bits corrupted by a faulty a network cable, as discussed in the sidebar "Case Study: File Corrupted During Replication".

New wrong data

If the acceptable formats (what is "good," or the allowlist) aren't fully specified in the application logic, the data you accept might fail further along in the execution stream. For example, edge cases could be missed if the system attempts to define only what is "bad" (a denylist).

Expired existing data

If fresh versions of the data don't arrive, existing data can become stale and the system may have to degrade gracefully. For example, cached information stops being valuable when the referenced objects change quickly, such as the addresses of healthy backends.

Your systems should test for, detect, and correctly reject all bad inputs. You should also regularly test your system's handling of bad inputs, including the effectiveness of its prevention and mitigation mechanisms.

Actionable Hardening

When your systems are hardened to bad data, they respond to implausible inputs by continuing to operate in the previous state or resorting to error handling techniques (cov-

ered in "Overload Management"). This resilience complements preparations for the failure scenarios covered in Chapter 16.

In several outages Google has experienced internally, if the system had simply continued doing what it was doing before and ignored new suspicious instructions from the control plane, the outage would have been averted.

Design expectations include:

- *Diff too large*: Design the system to recognize when the diff between the old and new data is unusually large, to perform additional checks on that diff, and to reject it if there's any doubt. For example, if quota limits or authorized user lists shrink or grow substantially, it may be safer to require additional indicators that the change is intentional.
- *Partial updates*: Design the input to make data completeness easy to verify by providing redundant information. For example, a data structure with a set of permitted identities could include a field with the expected size of that set.
- *New wrong data*: Design the input so that there's only one correct format, making it practical to define what "good" looks like and avoiding trying to define "bad." For example, use stable ordering for repeated fields and prohibit duplicates where appropriate.
- *Keep good data*: Design the system not to discard good data until necessary for reclaiming system resources or for compliance with data handling policies. For example, keep the previous version of data in memory so you can roll back to it with the change of a memory pointer rather than retrieving it from the original stable storage (presuming the older version even remains readable).

Hardened systems should also alert on the receipt of bad input. This should happen not only when manual action is needed but also to collect actionable data about trends and patterns in handling failures, to improve risk management.

Data Resilience

Resilience in data management is critical because data failures are categorically different from other types of failure. Chapter 16 covered how remediation can be much harder—or impossible—when data is lost and what your systems should do to reduce opportunities for data loss from many higher-probability failures. In this chapter, we focus on the higher-layer problem of data loss due to unexpected interactions or gaps between cooperating systems.

Your organization and systems probably manage user and infrastructure data using multiple storage systems, including physical storage, raw logical storage, distribution databases, and pipelines for derived data. Even with redundancy designed into every layer, you should prepare for some systemic risks of permanent loss of user data or your organization's data.

Here are some theoretical examples of lower-probability high-impact scenarios you should plan for:

- *Recovery of metadata loss* within a storage system—e.g., the knowledge of where the redundant data is and its criticality in enabling access to encrypted data bytes.
- *Recovery of encryption keys*, such as the data encryption keys (DEKs) or key encryption keys (KEKs) that implement the security controls safeguarding data bytes from unprivileged use. Even data restore processes require these.
- *Unintentional destruction of data* due to secondary effects of routine operations by lower-layer storage systems—e.g., when database or storage device replicas are removed from service without the semantic context that those are the copies for a mission-critical system.
- *Recovery of failures in data management controls* meant to limit data losses—e.g., when a "write-once" storage system loses the capability to block data overwrites.

The velocity of reaction to these failure scenarios is critical because of their potentially global breaking impact. Implementing all the relevant advice from Chapter 16 can significantly improve reaction time and lower the overall cost of cross-system data management risks.

This section presents recommendations that can help reduce the risk of data loss or delays in data recovery.

Deglobalize Data Loss

Any action on data—especially when humans are involved—should be considered a major threat to system integrity, which could manifest through potentially unintended global changes. Deglobalization of data operations is one way to limit impact. If you can build structural security barriers that restrict management actions on storage systems, then only smaller parts of the production environment can be updated at any time. This makes it virtually impossible for a single error or action by a human operator or an AI agent to cause an instantaneous, fleet-wide data disruption.

Here are some ways change management can help deglobalize data loss:

- *Rollouts for configuration and binaries* for storage systems (e.g., block storage nodes) should be restricted to a scope that prevents bad rollouts from affecting the entire fleet.
- *Mutating operations on nodes and devices* used by storage systems (e.g., OS upgrades, device upgrades) should be paused if global redundancy of datasets affected by the operation is insufficient.
- *Mutating operations on stored data* (e.g., immediate deletion of data blobs) should be limited in scope and ideally reversible.

More than one of these categories could coincide, so it's also important to manage change concurrency (see "Change Concurrency").

Once you define what you want to pursue, enforcement can progress in stages:

1. Provide observability into risky operations, to know how quickly they land across the global scope and if the expected safeguards are being used.
2. Deglobalize automation and operators, because those could repeat locally scoped operations many times, maybe without supervision, to effectively bypass deglobalization objectives.
3. Harden against elevated privileges, which can be inherently global and bypass any deglobalization constraints.

These scope reductions may introduce inefficiencies elsewhere, so you'll have to decide what's acceptable. For example, schema changes in the higher-level storage components may require substantially more storage in the lower-level storage systems, albeit intermittently.

Delete and Undelete Data Correctly

Make data deletion resilient by allowing only staged deletions, with built-in grace periods (like the "move to trash" function) that are not bypassable, except through completely separate, well-guarded processes dedicated to express requests by data owners or to your organization's specific needs.

Ensure that critical data management systems implement safety features, with safe defaults, to protect against accidental data loss caused by users. For example, critical deleted data could be kept for at least X days (within the retention commitments to data owners)—the opposite of setting a default expiration TTL for automatically deleting

ephemeral data. Note that there is a risk of accidentally setting a TTL on critical data; such situations should be detectable for confirmation of intent, or prohibited via code logic if appropriate (see "Input Validation").

Design expectations include:

- Mechanisms to prevent unintended data loss should be implemented, ranging from impact warnings for user actions to configurable and automated deletion protections at appropriate layers of the application and the storage infrastructure.
- Rapid data recovery should be supported through default soft-deletion mechanisms ("undelete"), extended retention windows for critical assets, and hardened protections that prevent the manual purging of deleted data until the system-managed recovery window expires.
- Systems should enable data recovery with minimal dependencies, ranging from basic integrated restore features to deletion-resistant backups and support for per-user or per-system reinsertion of restored data into live databases.

For the design expectations valuable for your organization, you can progress in stages:

1. *Define deletion safety guidelines.*

   At the organizational level, determine the safety measures appropriate for the data storage systems in broad use. Not all expectations will apply universally; guidance should be specific to each system, considering its nuanced purposes and constraints. Audit the available features regularly to adjust system-specific data protection policies and refine the general guidelines. Future data storage systems should follow the guidelines and be included in the audit.

2. *Remediate gaps with safety guidelines.*

   Systems should develop the missing safety features and/or change the defaults to enable data resilience, as well as provide observability into feature usage across datasets. Your organization can then apply broad mitigations (covered in "Safe Mitigations") to identify production assets that need attention.

Resilient data deletion can mean higher storage costs to accommodate the extra bytes staying around for longer. You should pick the most valuable places to introduce these overheads.

Independent Fallbacks for Critical Data

Loss of system metadata can be catastrophic if that system, as a result, cannot be re-assembled to resume its critical functions. That's why this type of data—critical data—requires defense in depth.

If critical data uses backups, there should be no common dependencies (shared fate) between the systems using that data and the processes for restoring it. If common dependencies exist, there's a chance that it won't be possible to complete a restore when needed because both the system and its recovery are affected by the same failure. See "Dependency Management" and "Failure domains" for more on this topic.

Let's consider two alternatives for recovering from storage metadata corruption:

Recovery via underlying storage backups

If storage system X loses its metadata, and X relies on storage system Y for its underlying storage, restoring system X's metadata requires system Y's metadata to be intact. If system Y was also damaged, you might recursively have to recover system Y's metadata first, hoping that the system's underlying storage (perhaps in storage system Z) is intact and has data restores configured and tested. This might work, but it will take a long time, which might be unacceptable in the midst of a global outage.

Recovering from independent fallback backups

The shortcut (when faced with that lower-probability combination of misfortunes) is to back up metadata to an independent storage system whose entire stack is independent of the primary storage system. As we covered in "Managing Fallbacks", you can choose the appropriate degree of independence. For example, a system using PostgreSQL databases could store its database metadata in a separate PostgreSQL instance, or in a different database management system that is architecturally and organizationally distinct from PostgreSQL.

Both approaches have pros and cons. Where the trade-offs are justified for the system being designed, ideally you'll use both for maximum resilience.

Fallback backup systems can potentially be reused for various cross-system resilience needs. For example, encryption keys and related data verification mechanics must not be susceptible to complete loss, as that would cause permanent data loss. With sufficient security precautions, you could preserve the decryption keys needed to recover backup

data and the encryption keys needed to re-encrypt the restored data so that it is readable only by its intended system.

Fallback backup systems should have appropriate protections, so you can rely on their resilience. For example, read-only backup systems should ensure that any read-write capabilities cannot destroy the backed-up data.

Recover Fast from Data Loss

Some of the scariest imaginable disasters occur when both data and metadata are lost, or when multiple storage layers are severely disrupted. This can apply across block-level and file-level storage systems, databases, and higher-level specialized data stores.

If there's even a recovery plan to execute, recovery speed becomes critical. This can be a matter of organizational survivability, and failure to recover can have a significant impact on users.

Design expectations include:

- Storage systems should define a plan to be able to recover, at a minimum, enough that users' data can start being restored from backups. The plan's feasibility should be validated for at least one failure domain (see "Recovering Failure Domains").
- Storage systems should estimate the time required to recover to the point where most user data can be restored, especially for systems that provide users with native backup solutions. Validation of this should already be covered by storage systems, but if not, remember that practicing recovery can uncover surprising roadblocks, such as struggling to obtain sufficient resource quotas for all user data.
- Storage systems should be designed with a clear strategy for whether to begin responding with no user data (while at least having the opportunity to explain the situation and possibly restore and reconcile later) or to remain unavailable until the full dataset has been recovered.

Because data loss can have unexpected secondary effects on operational capabilities, the risk of recovery plans failing or RPO/RTO/RWO targets (discussed in "SLOs") not being met is elevated. This is expected, and the one definitive way to discover and mitigate those risks ahead of time is through qualification in a realistic setting.

Qualification

As teams continue development, they need to ensure that changes continue to meet requirements and remain backward compatible. Qualification of resilience is critical to achieving this. You can think of qualification as validating your ability to maintain and grow users' faith in your organization's products and mission. Here are some areas of focus:

Freshness

Tests should be updated over time, as requirements change in response to lessons learned during outages. This applies to all testing categories and mechanisms covered in Chapter 14 and is applicable to resilience designs implemented by your systems and organization.

Realism

Validation of resilience focuses on observing systems and teams in sufficiently realistic yet still controlled circumstances. To break this down further, you want to cover workflows within and across systems and teams when they are operating in normal, emergency, and even disaster modes (covered in "Incident Modes"). As mentioned in earlier chapters, resilience cannot be assumed if the expected behaviors haven't been validated *realistically* and *recently enough*. Appropriately, use a mix of pre-production and in-production environments.

Theoretical

Where practical validation is infeasible, theoretical exercises can still provide substantial value. For example, performing a realistic test of recovery from global data loss can subject your organization to existential risk. Instead, you can conduct a scenario-based "pretend" test (see "Disaster Role Playing" in Chapter 28 of the first edition of this book) that runs through all the relevant interactions and activities, leaving out the dangerous operations. It's a bit like *Dungeons & Dragons*, except players also know the "magical" IMAG protocol (discussed in "Incident management"). Then, following the theoretical exercise, the "players" can practice further by running safer variants of commands that surfaced during that exercise.

Participation

If qualification activities are optional, they likely won't be sustained. Instead, aim for what can be made cheap, repeatable, and automated, so that the qualification

process is in place and focused on both discovery of novel failures and validation of remediated risks. Where appropriate, automate the scheduling of qualification activities, as well as reporting on their regularity and results. Automate all activities that don't require human involvement. Where there are cross-team dependencies for qualification, ownership and responsibilities should be clear and acknowledged.

Environments and Shifting Left

Teams should maintain environments for all key reliability and resilience test types to ensure adequate coverage of their systems. The organization should typically make provisioning these environments part of the testing strategy for all developers.

System owners should then adapt each qualification activity to the appropriate environment. By using multiple environments, you balance safe risk allocation with gradual confidence building:

- Not all defects can be caught in earlier qualification phases (e.g., integration tests), because the fidelity of the systems under test is usually insufficiently representative of the system's realistic ("lived") experience. However, the more defects you catch early, the fewer escape to the later phases of qualification, where environments are larger and debugging is more expensive due to many changes being tested together.
- Defects can be caught more quickly after they are introduced in the qualification phases where testing can run more frequently. This happens more in the earlier phases than in the later phases, because testing is cheaper there (e.g., unit tests).
- Defects found in later phases of qualification should be mostly novel discoveries or surprise regressions not caught in earlier phases, with the majority of tests passing as a realistic confirmation of behaviors tested earlier.

After you reach a steady state of qualification coverage, the point of expensive tests in more realistic environments is to confirm that the cheaper tests in less realistic environments continue to represent and successfully qualify all important resilience behaviors. More realistic environments can also be used to validate human response plans, ensuring that operators can perform the appropriate actions when the system ends up outside its designed norms. It's important to discover novel failure scenarios.

To reach that steady state, use only pre-production environments to test expected resilience behaviors that don't require the full complexity of live production. This will help you establish a safety track record for the tests, which reduces the probability that the same tests will cause outages if they're repeated in production.

Realistic Environments

Qualification of resilience should at some point use a test environment permitting setup of real and complete conditions for targeted failure scenarios. We'll provide a few examples here, but it's up to you to define what that looks like. Qualifications performed outside of these defined conditions are insufficient for building trust in the system's resilience capabilities.

For example, to confirm user-visible recovery after the loss of several table rows, you must observe that your live system instance—the one that's currently serving errors to users—resumes its functions after you restore a backup or snapshot and copy the missing table rows from there into the live database. By actually deleting table rows in such a test, you definitively exercise many parts of the expected recovery workflow. Make sure that the test succeeds regularly in non-production environments before repeating it in production, and that it uses test data created and added in production specifically for this purpose.

As another example, to qualify a "rollback" of a production router by reinstalling and reconfiguring its OS over its standard out-of-band network interface, ensure that your network isolates that router such that it cannot communicate with anything using its normal network ports. Then, after the qualification procedure completes, deisolate the router and confirm it resumes normal functions.

It can be expensive to build and maintain environments for realistic testing. At Google, we find multiple uses for these environments, which increases our ROI. Building and testing system resilience in these environments helps offset the costs of the physical resources and operational toil of maintaining extra instances of every system that makes up the ecosystem whose resilience we test.

Practical Validation

Regardless of the test environment you select, or whether the exercise is theoretical, validation should be performed regularly, at a frequency commensurate with the value and costs in each specific case. It can be achieved through a single test or a combination of multiple tests, but only the end-to-end aggregate workflows that accomplish recovery outcomes count as practical validation.

Design expectations include:

- Resilient behaviors (expected reactions and recovery actions in response to changing production conditions) should be observed in practice. The more critical system components and/or interactions there are, the more coverage and realism their validation deserves. Realism includes setting the conditions that would evoke a resilience response from a real system or its high-fidelity copy, and confirming that the response meets expectations.
- Teams should validate their operational abilities in all modes of operational disruption (see "Incident Modes") applicable to their systems, prioritized commensurately with tracked risk assessments. This includes setting up the work environment (workstation, shell or browser access to tools, etc.), locating and accessing relevant standard operating procedures (playbooks), and executing actions that investigate or modify a realistic instance of the system.

For example, Google's resilience engineering team would prepare a realistic environment that simulates disaster conditions, tuned for practically validating a large set of disaster recovery scenarios, as well as testing operations continuity across incident responders. Within this environment, we would isolate systems from the rest of production (e.g., making DNS and load balancing unavailable, among many other critical dependencies), setting the stage for uniquely valuable DiRT tests (see "Disaster Recovery and Resilience"). The on-callers participating in those tests could then practice recovery procedures in conditions matching the outage conditions they wanted to validate against. In tests simulating a substantial network outage, for example, this might enable the discovery of localized cyclic dependencies and behaviors that made certain systems completely unusable (and were easy to fix).

As another example, Google's teams occasionally handle imminent power loss in datacenters. In these situations, emergency power-off procedures for affected clusters rely on the safe shutdown of local infrastructure systems. Joint emergency training for datacenter and SRE teams involves real cluster reboots, under controlled but absolutely real conditions.

These examples are meant to show how, due to the complexity of Google's technical stack, systems and teams cannot rely solely on designs or theoretical exercises. Instead, we aim to develop maturity demonstrated through continuous validation of resilience capabilities. That maturity can be represented by an expanding set of clear statements in the form: "Workflow X is resilient to failure Y, as measured by Z."

What to validate (the sets of X, Y, and Z) can be defined when systems proactively and regularly perform resilience scenario analysis, triage identified gaps (using approaches

such as STPA and CAST), and select which of them need coverage, even in emergency mode.

Emergency Mode

Incident response, covered in Chapter 10, focuses on mitigating failures. But if failures begin to inhibit incident response itself, things stop being normal for responders. At Google, we call these conditions *emergency mode*. To prepare for these conditions, rather than trying to anticipate every possible permutation of failures, we've designed a set of flexible response capabilities that have broad applicability.

For example, a responder typically uses security credentials, network access, authorization proxies, communications, incident management software, and other tools to triage outages and execute mitigations (e.g., expedited rollbacks or data pushes). When some of these components break (e.g., issuing new credentials), parts of the incident response break, too. The affected responders then need alternative ways to carry out their operational tasks. These responders are now in emergency mode.

The overall objective is to ensure incident response can continue, even when core services and infrastructure—including the tools for recovery—are failing, thereby preventing a complete lockout. This can be achieved by creating distinct, highly resilient fallback mechanisms for critical capabilities such as access, communications, approvals, and other response CUJs.

Designing for emergency mode involves organization-wide and system-specific investments. Generally, the larger the outage, the less infrastructure remains available to support incident response, the more responders are forced into emergency mode, and the more effectively they need to collaborate to mitigate that outage. In these conditions, responders need clear and composable solutions for common parts of incident response CUJs.

Incident Modes

Incident modes are a tiered approach to incident response that categorizes operational circumstances based on the severity of a failure (Figure 17-2). A key consideration is how that failure affects the tools and infrastructure used to manage the incident itself (Table 17-1).

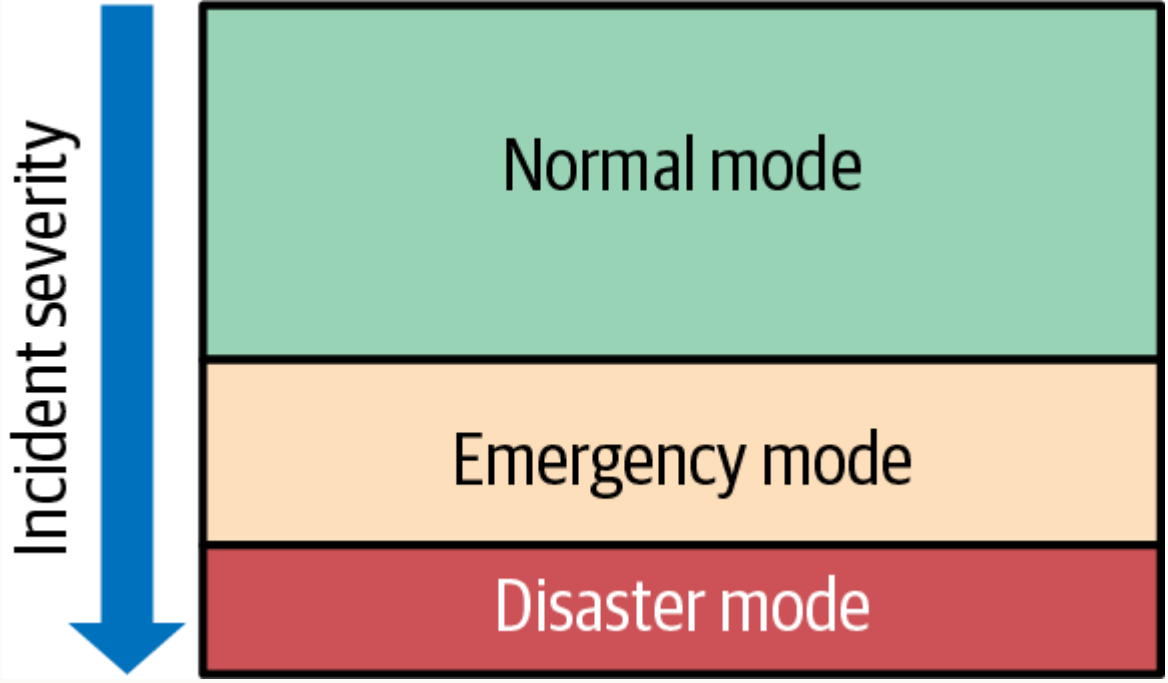


Figure 17-2. Incident severity usually grows as incident mode degrades toward disasters

Table 17-1. Comparison of incident modes

Mode	Key characteristics	Dependencies and tools	Target responders
Normal	The standard mode of operations for everyday incidents, used when failures are routine and do not stop the incident response process.	Uses high-dependency systems, standard interfaces, and rich features.	Everyone, most of the time.
Emergency	Failures inhibit the use of normal incident response tools, but it's possible to anticipate and plan for these failures by building robust fallbacks that allow responders to more or less follow the standard incident response process.	Uses low-dependency fallback systems, which may maintain the same standard interfaces familiar to responders.	Everyone, most of the time, but with possible capacity or security restrictions (most responders who operate in normal mode can switch to emergency mode fallbacks with minimal training and usually on the spot).
Disaster	The last-resort mode for catastrophic events, where responders rely on specialized skills and tools in the absence of fundamental distributed services (e.g., global network failure).	Uses zero-dependency components and adapts available interfaces to work together (e.g., command-line tools, direct network access).	Small specialized operations teams with rigorous training for required highly technical skills.

This chapter focuses primarily on emergency mode, because emergencies have much higher probability than disasters and deserve priority in engineering investment. As mentioned earlier, you should still make plans for disasters and allocate sufficient engineering capacity to manage them.

Incident modes can provide distinct roadmaps for both system developers and incident responders, helping them maintain operational capacity during failures. For leadership, understanding incident modes can help sustain investment in those roadmaps, ensuring they remain effective over time.

For System Developers

Emergency mode preparedness guides system design to ensure resilience by creating layered, low-dependency fallbacks. System developers should:

Design for low dependency.

Create simplified versions of systems expected to still work in emergency mode, using the smallest set of required dependencies spread across the smallest set of failure domains. For example, it's ideal if the whole system (with dependencies) fits onto one machine and doesn't have RPC dependencies outside that machine. Not every system needs such a fallback, given the implementation constraints. Prioritize the smallest number of critical systems necessary for bootstrapping your organization's technical stack to the point where the majority of engineers can play a role in recovering the totality of your systems.

Maintain interface consistency.

Ensure that normal and emergency mode components use compatible interfaces, to optimize transitions for responders. For example, to switch to a fallback system, the responder will need to change the address of the system they use but shouldn't need to change the core elements of their workflow.

Define fallback capabilities.

Contribute documentation modules for fallbacks available for the system's functions and indicate how to correctly activate low-dependency features in the adopted fallback solutions.

Support mode transitions.

Systems should be able to detect failure, suggest to the user how to activate fallbacks or low-dependency modes, and show clear indicators when they are operating in emergency mode. For example, error messages could include actionable guidance for switching to a fallback. This can substantially reduce cognitive load on responders.

Retain usability during failures.

Systems should remain stable during failures that disable features (e.g., through graceful degradation of GUI or CLI behaviors). Avoiding complete user interface failure and reducing responders' cognitive load improves incident management efficiency at scale over time.

Retain security posture.

The security posture of system components *must not degrade* as you move from normal to emergency to disaster mode. This discourages attempts to bypass security by intentionally breaking normal systems.

For Incident Responders

The tiered incident modes can serve as a framework for cognitive simplification, providing responders with a structured decision tree to navigate complex outages. Incident responders should:

Follow decision flow.

Upon incident activation, responders should evaluate the situation through a principled sequence to determine the appropriate operational mode:

1. Is the normal workflow available? (This evaluation happens implicitly.)
   - *Yes*: Remain in *normal* mode and focus on mitigating failures.
   - *No*: Proceed to the next evaluative step.
2. Is the system or procedure providing guidance to use emergency fallbacks?
   - *Yes*: Transition to *emergency* mode by substituting the specified fallbacks into the familiar workflows.
   - *No*: Wait for workflow owners to fix it, or, if you cannot wait, transition to *disaster* mode by coordinating a specialized response team of available production experts with the most relevant experience and knowledge.

Embrace fallbacks.

Responders should view mode transitions as a simplified, binary choice to switch to fallbacks. This mindset ensures efficacy during even the most extreme failure scenarios.

Prefer guidance over automation.

During an outage, automated systems might not see the full system state or the potential hazardous side effects of their actions. Systems should favor providing actionable guidance to human responders rather than fully automating response. Responders should validate recommendations before acting on them.

Anticipate mixed modes.

Production environments are complex; responders may operate in multiple modes simultaneously, as individual tools and workflows transition based on the degradation of their specific dependency chains. For example, to mitigate outages of Google Chat, responders need to use secondary communication tools as fallbacks, but all other response capabilities might still be in normal mode.

Consider procedures for an emergency rollback or an emergency upsize of capacity. These are generic safe mitigations critical to preventing and mitigating outages, used to expedite changes when time is of essence. Normally, these actions might work with a few simple actions in a management app or a web interface (this is true at Google). However, in emergency mode, the responder might see an error message leading them to documentation for crafting an equivalent CLI command and running it in the appropriate environment. While the CLI differs from the app, the process remains conceptually standard and familiar to the responder. The responder can follow that guidance, and with a little extra delay, initiate the emergency action.

For Leadership

Understanding emergency mode can help organizational leadership prepare for risks during complex outages and sustain that preparedness over time. Leaders should focus on:

Broad mitigations

Teams should be able to perform relevant mitigations already prepared for normal mode in affected critical components, and in a timely manner. For example, following a large-scale disruption, systems might need to move quickly into healthy failure domains, and subsequently to fully recover the vacated failure domains (e.g., to avoid data loss or reclaim lost capacity).

Resilient workflows

Teams and systems should identify and fix regressions or gaps in their workflows for emergency mode, and report gaps in how other teams integrate and document these workflows. A common regression is when low-dependency fallbacks develop cyclic dependencies on their main systems (for which they are the fallback), so the fallbacks become unreliable. For example, when upgrading a system's dependency to a new technology, teams might configure both the main and the fallback with the same new backends, forgetting that the fallback requires a distinct low-dependency variant. These changes must be detected and undone promptly.

Response CUJs

> Leadership should define, select, and pursue CUJs essential for incident response in emergency mode. As a result, teams and systems should be able to participate effectively—and on a short timeline—in mitigations up to company-wide scale during prolonged or permanent failures of the underlying infrastructure.

Another role of leadership is to compensate with discipline (e.g., policies and conformance checks) for the natural tendency of people to deprioritize lower-probability risks, even if their impact is perceived as high.

Response CUJs

You can think about incident response in terms of two high-level influences:

- What people need to do
- What systems supporting people can do

Together, these determine a minimal set of *response CUJs* that are high-level, sufficient, and practical. Response CUJs must be composable, to enable arbitrary response paths toward mitigation and dependable and redundant, to prevent response lockouts. Each CUJ must be translated into one or more variants, implemented by concrete procedures and systems and with minimal divergence from what responders normally do.

For example, during an outage, responders might need to provide quick internal updates to executives. That ability could be constrained if the primary or secondary internal communication tools are down, or if responders are not able to quickly figure out the outage's impact. Therefore, we should have multiple solutions for the following CUJs: inspecting the production state of components affected by the outage, and communicating and collaborating among responders and with executives. Those solutions might include systems for inspecting production state and documented workflows for performing the inspections in emergency mode.

Table 17-2 shows categories of response CUJs we consider valuable in emergency mode. The prefixes can be used when creating IDs for the response CUJs.

Table 17-2. Categories of response CUJs

Prefix	Category	Description
comms	Establishing communications	Responders can connect and coordinate with each other, even when normal tools like Chat/Meet are down.
imag	Coordinating response	Responders can manage the incident lifecycle (declaration, roles, tracking, handoff) using the IMAG protocol.
dash	Informing stakeholders	Responders can provide timely updates internally (to executives) or externally (to customers).
reach	Accessing systems	Responders can reach system components (hardware devices, service endpoints) that need fixing or have tools for incident response or recovery, even if normal network/office paths are unavailable.
doc	Accessing documentation	Responders can access critical documentation (procedures, guides), even if standard Google Docs or Markdown access is disrupted.
priv	Activating permissions	Responders can gain the necessary production privileges to modify systems, including mode-appropriate multiparty authorization flows if needed.
dbg	Debugging systems	Responders can investigate system behavior, view monitoring data, and read logs to diagnose issues during an outage.
fix	Modifying systems	Responders can make production changes, including rollbacks, resource adjustments, configuration changes, and load-balancing changes.

Within each CUJ category, you can define actual response CUJs and their essential capabilities and attributes, including which systems implement them. Here's a short example showing two actual CUJs for the establishing communications ("comms") category:

- *CUJs*:
  - Real-time text messages among responders ("comms-text").
  - Real-time audio/video conversations among responders ("comms-av").
- *Normal systems*: Google Chat, Google Meet, Gmail
- *Fallback systems*: IRC, direct phone calls (cellular)

It can be helpful to map the response CUJs to systems and procedures and the supported incident modes (N for normal, E for emergency). Tables 17-3, 17-4, and 17-5 are examples of what this might look like at Google. For brevity, we've included only three systems (the columns to the right in those tables); in reality, the number of systems can vary. The systems shown in these examples are:

Chat

Google Chat, used for communications using text and images

Meet

Google Meet, used for voice and video communications

Fallback

Represents our lower-dependency solutions designed for emergency mode

IRM

Google's internal incident response management system for the outage lifecycle

Docs

Google Docs, used for collaborating on dynamic unstructured information

GCS

Google's public Cloud Status site

PSH

Google's public Personalized Service Health site

Table 17-3. Response CUJs for communications

CUJ ID	Description	Chat	Meet	Fallback
comms-text	Real-time text messages among responders.	N		E
comms-av	Real-time audio/video conversations among responders.		N	

Table 17-4. Response CUJs for incident management

CUJ ID	Description	IRM	Docs	Fallback
imag-declare	Declare an incident and escalate it appropriately.	N		E
imag-handoff	Transfer roles and context to incoming responders.	N	N	E

Table 17-5. Response CUJs for informing stakeholders

CUJ ID	Description	IRM	GCS	PSH
dash-internal	Broadcast status updates to affected internal teams.	N		
dash-external	Update external users about incident status.		N	N

How to implement these (and other) CUJs for emergency mode is up to you. For example, you could decide to use telephone calls for high-bandwidth discussions, IRC channel messages to publish current IMAG role assignments, and an external company's workspace services for keeping notes.

The best-case scenario is that your normal tooling continues to support these CUJs. However, you may need to do significant engineering work to ensure resilience when "everything is down." You'll need to decide how much to invest in additional robustness for each response CUJ.

Training and Practicing

People should practice their ability to use response CUJs in emergency mode. While the following considerations apply in any incident mode, emergency and disaster modes leave much less margin for error. Because people rarely encounter emergency mode, their ability to respond might not develop naturally and will degrade over time. Therefore, you should pay attention to how prepared your incident responders are.

Teams should be prepared to respond to incidents outside of normal conditions. Procedures and system behaviors might differ, for example because fallback implemen-

tations often limit capabilities to only the most critical ones. When we know that on-callers can access a broken system even if the central login or proxy systems are unavailable, we can expect outages to have shorter mean time to mitigation (MTTM).

Additionally, teams should maintain sufficient collective expertise for their systems to compensate for individuals changing projects or teams or going on vacation, office-wide local personnel outages, etc. For example, if only two people have in-depth knowledge of critical technical nuances, what will happen if one goes on vacation and the other gets sick? Considering such possibilities is part of your risk management efforts.

As you prepare your team, it's important to understand the distinction between training and practicing. For example:

- *Training* gives you an *understanding* of what CUJs are available and how they work.
- *Practicing* gives you *experience* performing CUJs before you need to use them during outages.
- *Not all training involves practicing*—e.g., online courses always offer training but only sometimes offer practicing.
- *Not all practicing requires training*—e.g., a workflow could encapsulate all necessary understanding but still require practicing to rule out person-specific setup risks.

Training and practicing for emergency mode have a higher cost-value ratio (this is natural). Therefore, you should prioritize response CUJs that are the most frequent and deviate the most from their normal mode variants. Here are some ideas for when to pay those costs.

Training may be needed when:

- New tools are introduced, replacing parts of everyone's normal response processes.
- Whole new processes are introduced, changing everyone's starting point.
- There is evidence that responders are insufficiently trained on the existing procedures and tools.

Practicing might be prioritized as follows (across all categories of response CUJs):

1. Core actions prerequisite to fix production ("reach," "comms," "priv," "doc," "imag")
2. Common actions required to fix production ("dash," "dbg," "fix")
3. Bespoke actions specifically developed to fix only your system

For example, it should be easy to see that to investigate, debug, and fix a server in production, we first need to reach into production, assess the incident state, communicate

among responders, obtain sufficient privileges, and access contextual documentation. Therefore, those core actions should be practiced first, to provide a solid foundation.

It can be hard to make practicing sustainable until it can be made cheap and frictionless. It will help if responders who routinely demonstrate their ability to perform certain emergency mode actions aren't required to train on and practice those actions. Instead, they should focus on response CUJs that are becoming stale or have never been touched.

Outlook

We've covered a lot of ground in this chapter. Before we wrap up, we want to provide you with a few helpful pointers.

Drivers and Costs

Delivering resilient systems can be an expensive undertaking, and the act of negating future threats to these systems may make resilience improvements difficult to demonstrate (you're proving a negative). However, don't let this dissuade you from action.

In "Higher and Lower Probability", we discussed prioritizing higher-probability and lower-probability failures. Focusing on higher-probability failures first generates improvements that are easier to demonstrate (e.g., by reducing or eliminating the impact of those failures), which can increase your organization's willingness to invest in addressing lower-probability but still impactful threats. As a general guide:

- First, address the failures you've been encountering frequently and for which you can estimate the associated costs.
- Then, identify the lower-probability scenarios that you absolutely cannot afford to experience, and make plans that can "move the needle."

Don't be surprised if resilience initiatives compete for prioritization with your organization's other priorities, including business growth, user satisfaction, and security improvements.

If your organization is subject to industry regulations, you may find that business continuity and resilience requirements oblige you to demonstrate resilience. These regulatory requirements need to be translated into concrete expectations for resilience. Once you've clarified those expectations, the ideas in this and other chapters can help you identify appropriate approaches.

Taking a Snapshot

To make resilience practical, start by taking a snapshot of the resilience you already have and then defining your resilience strategy going forward.

Ask yourself these questions:

1. How does the understanding of resilience differ across people on my teams? Are we all aligned on resilience well enough to make consistent choices effectively?
2. In what ways does our system's current resilience posture match or mismatch user expectations about failures?
3. What is our strategy for selecting which resilience improvements to prioritize?

Check that your resilience strategy covers the following areas:

1. Mission-critical goals the system must support
2. Failure scenarios most likely to disrupt those mission-critical goals
3. Technical solutions for handling failures at acceptable costs
4. Validation plans for the durability of failure-handling solutions

Once your engineers and leadership are equipped with these details, you can start to iterate through practical improvements.

Making It Practical

You'll need a systematic approach to exploring the expected behaviors of your organization's systems and users, then deciding whether to adjust those expectations or the behaviors themselves:

1. Define a rubric for what could go awry, so that expected behaviors can be evaluated and prioritized according to the dimensions in this rubric. For example:
   - Global versus localized impact of failure
   - Fail-open or fail-closed reaction to failure
2. Start with areas where you have the least justification for not having expected behaviors that could prevent users from being impacted.
3. Check for dependencies that cannot be removed by system designers; the impact of failures in those dependencies cannot be ignored.
4. Check for alignment with business advantage, for the business or competitive edge, or existential protection.

5. Check for differences in improvement strategies between greenfield and brownfield systems, as the approaches might differ.

After you stack-rank the findings, don't ignore any dimension entirely. Even for categories where you assess the ROI of resilience improvements as lower, make sure to confirm that through a small sample of engagements with systems in those categories.

Failures Without People

One ambitious goal for the future of resilience is removing engineers from the loop of handling routine failures in live systems. This has several valuable effects:

- For outages, it typically speeds up mitigation by enabling faster execution, fewer mistakes, and more predictable outcomes.
- For engineers, it typically reduces toil represented by failure scenarios that are simple enough and consistent enough to be safely automated.
- For reliability and resilience, it keeps investments aligned toward innovation.

With advancements in AI, agents can start to apply safe mitigations, and perhaps mature to assist with more complex outages. This would free up engineers to focus their time and attention on the most critical and creative work: handling new design challenges, novel outages, and failure combinations we haven't seen before, allowing our engineering expertise to continue to evolve and enabling us to further develop, enhance, and mature AI.

Conclusion

Remember that user expectations of reliability and resilience are ultimately what matter. If your system does not demonstrate viability after failures, users might work around its limitations…or switch to a more competitive alternative.

Every resilience improvement has costs but only *potential* future benefits. Your challenge is to prioritize the most valuable resilience efforts and determine when progress is sufficient to stop investing in one area and move on to the next. Keep in mind that:

- Every change you implement to help systems stay reliable beyond their current operating parameters can be a small but durable structural improvement.
- As new resilience behaviors get integrated into systems, those systems become more reliable. You can then identify the next resilience risks worth mitigating permanently,

and the cycle continues.

Every resilience improvement counts; small steps add up. That's how investing in resilience advances your organization's reliability frontier.

1 In terms of what users can assume will be handled versus what they are expected to act on in possible system failures.

2 A risk register is a document recording the probability, potential impact, risk owner, and mitigation strategies for each tracked risk.

3 Anti-affinity is the strategy of keeping components separated to maximize fault tolerance and limit blast radius (e.g., not putting two database replicas in the same rack, or even the same datacenter).

Previous chapter
< V. Resilient Products and Emergent Reliability

Next chapter
18. Platform Engineering in SRE >